



CodeExpert
Technology, Gadhinglaj.

Mastering

Python Programming & Industry Applications



A complete Python Guide

with industry applications, coding techniques, and interview simulations

Mr. Ketan Balappa Gorule

Table of Contents

Part 1 - Fast Track Power Revision: Introduction to Computer Programming

Chapter 1: Getting Started with Python

- Lesson 1: Introduction to Python Programming
- Lesson 2: Data Types in Python
- Lesson 3: Indexing & Slicing
- Lesson 4: Operators in Data Types
- Lesson 5: In-Built Functions & Methods

Chapter 2: Python Control Flow

- Lesson 1: Statements, Indentation & Conditionals
- Lesson 2: Loops & Iterations
- Lesson 3: Conditional & Infinite Looping

Chapter 3: Object-Oriented Programming

- Lesson 1: Custom Functions in Python
- Lesson 2: Advanced Looping Concepts
- Lesson 3: OOP in Python
- Lesson 4: Exception Handling in Python

Chapter 4: Bonus Chapter – Advanced Python Concepts and Competitive Coding

- Lesson 1: Recursion in Python
- Lesson 2: Pattern Problems
- Lesson 3: Build Your Python Logic Like a Pro

Part 2 - Practical Industry Applications & Interview Simulations

FinTech

- Industry Overview & Python Applications
- Applied Industry Scenario: Personal Finance Tracker
- Applied Industry Scenario: Loan Eligibility Predictor
- Applied Industry Scenario: Bank Transaction Monitoring System
- Crack the Industry: Insights, Strategies & Wrap-Up

HealthTech

- Industry Overview & Python Applications
- Applied Industry Scenario: Patient Record Management
- Applied Industry Scenario: Health Recommendation
- Applied Industry Scenario: Medicine Stock Management
- Crack the Industry: Insights, Strategies & Wrap-Up

E-CommerceTech

- Industry Overview & Python Applications
- Applied Industry Scenario: Shopping Cart System
- Applied Industry Scenario: Order Tracking
- Applied Industry Scenario: Discount Calculator
- Crack the Industry: Insights, Strategies & Wrap-Up

ManufacturingTech

- Industry Overview & Python Applications
- Applied Industry Scenario: Inventory Management System
- Applied Industry Scenario: Defect Detection Logger
- Applied Industry Scenario: Production Line Status Monitor
- Crack the Industry: Insights, Strategies & Wrap-Up

MediaTech

- Industry Overview & Python Applications
- Applied Industry Scenario: Content Recommendation System
- Applied Industry Scenario: Video Metadata Organizer
- Crack the Industry: Insights, Strategies & Wrap-Up

EdTech

- Industry Overview & Python Applications
- Applied Industry Scenario: Online Library Management System
- Applied Industry Scenario: Quiz Platform with Automatic Scoring
- Crack the Industry: Insights, Strategies & Wrap-Up



CodeExpert
Technology, Gadhinglaj.

Lesson 1: Introduction to Python Programming

1.1 What is Python?

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. It is widely used in web development, data science, automation, artificial intelligence, and more.

1.2 Key Features of Python

- Simple and Readable Syntax – Python code is easy to write and understand.
- Interpreted Language – Python does not require compilation before execution.
- Dynamically Typed – No need to declare variable types explicitly.
- Extensive Standard Library – Includes built-in modules for various functionalities.
- Platform Independent – Python code can run on multiple operating systems.
- Strong Community Support – A large developer community with extensive resources.

2. Overview of Integrated Development Environments (IDEs)

2.1 What is an IDE?

An Integrated Development Environment (IDE) is a software application that provides tools to write, test, and debug code efficiently.

2.2 Popular IDEs for Python Development

- PyCharm – Feature-rich IDE for professional developers.
- VS Code – Lightweight, highly customizable, and supports extensions.
- Jupyter Notebook – Interactive coding environment used for data science and research.
- Google Colab – A cloud-based IDE for running Python code in an online notebook format.

3. Getting Started with Google Colab

3.1 Why Use Google Colab?

- No Installation Required – Runs entirely in the browser.
- Free Access to GPUs and TPUs – Useful for deep learning and machine learning tasks.
- Collaboration Features – Allows multiple users to work on the same notebook.

3.2 Creating and Managing Notebooks

To create a new notebook in Google Colab, follow these steps:

1. Go to colab.research.google.com.
2. Click on "File" → "New Notebook".
3. Start writing Python code in the provided code cells

4. Writing Code in Google Colab

4.1 Navigating the Google Colab Interface

The Colab interface consists of:

- Code Cells – Where Python code is written and executed.
- Text Cells – Used to add comments and explanations.
- Toolbar & Menu – Provides file management, execution controls, and notebook settings.

4.2 Writing and Running Python Code

Type a Python command in Colab inside a code cell and press Shift + Enter to execute it.

```
print("Hello, World!")
```

Output:

Hello, World!

5. Editing and Running Code in Colab

5.1 Code Cells vs. Text Cells

- Code Cells – Used for writing and executing Python code.
- Text Cells – Used for writing markdown text, explanations, and comments.

5.2 Modifying Python Code in Colab

To modify a code cell, click inside the cell, edit the code, and re-run the cell using Shift + Enter.

5.3 Executing Code Cells

Code cells can be executed one at a time or all at once using "Run all" in the toolbar.

```
x = 10
```

```
y = 5
```

```
print(x + y)
```

Output:

15

6. Basic Data Types in Python

6.1 Overview of Fundamental Data Types

Python has several built-in data types, including:

- Integers (int) – Whole numbers (e.g., 10, -3, 1000)
- Floating-Point Numbers (float) – Decimal numbers (e.g., 3.14, -2.5)
- Strings (str) – Sequence of characters (e.g., "Hello", 'Python')
- Booleans (bool) – Logical values (True, False)

6.2 Examples of Data Types

```
num_int = 10 # Integer
```

```
num_float = 3.14 # Float
```

```
text = "Python" # String
```

```
is_valid = True # Boolean
```

```
print(type(num_int)) # Output: <class 'int'>
```

```
print(type(num_float)) # Output: <class 'float'>
```

```
print(type(text)) # Output: <class 'str'>
```

```
print(type(is_valid)) # Output: <class 'bool'>
```

7. Variables and Naming Conventions

7.1 What are Variables?

A variable is a named storage location for data in Python. It allows values to be assigned, modified, and reused throughout a program.

7.2 Rules for Naming Variables

- Must start with a letter or underscore (_).
- Cannot begin with a number.
- Can contain letters, numbers, and underscores.
- Case-sensitive (age and Age are different variables).
- Should be meaningful (student_name is better than x).

7.3 Variable Declaration and Assignment

```
name = "Alice" # String variable
```

```
age = 25 # Integer variable
```

```
height = 5.7 # Float variable
```

```
is_student = True # Boolean variable  
print(name) # Output: Alice  
print(age) # Output: 25  
print(height) # Output: 5.7  
print(is_student) # Output: True
```

8. Practicing Variables in Colab

8.1 Assigning and Printing Variables

```
city = "Bhubaneswar"  
country = "India"  
population = 900000  
print("City:", city) # Output: City: Bhubaneswar  
print("Country:", country) # Output: Country: India  
print("Population:", population) # Output: Population: 900000
```

8.2 Reassigning Variables

Python allows variables to be reassigned at any time.

```
x = 10 print(x) # Output: 10  
x = "Python" print(x) # Output: Python
```

8.3 Combining Strings and Variables

```
name = "John"  
age = 30 print(name + " is " + str(age) + " years old.")  
# Output: John is 30 years old.
```

9. Summary

- Python is an interpreted, high-level programming language known for its simplicity and readability.
- IDEs like Google Colab provide a cloud-based environment for writing and running Python code.
- Python uses code cells for execution and text cells for documentation in Colab.
- Python supports multiple data types including integers, floats, strings, and booleans.
- Variables store data and follow specific naming conventions for clarity and consistency

Lesson 2: Data Types in Python

1. Introduction to Data Structures in Python

Python provides several built-in data structures that help in organizing and storing data efficiently. These include integers, floats, strings, lists, tuples, sets, and dictionaries. Understanding these data types is essential for writing efficient programs.

2. Numeric Data Types

2.1 Integer (int)

An integer is a whole number, positive or negative, without decimals

```
x = 10 # Integer
```

```
y = -5 # Negative Integer
```

```
print(type(x)) # Output: <class 'int'>
```

```
print(type(y)) # Output: <class 'int'>
```

2.2 Floating-Point Number (float)

A float represents a number with a decimal point or in scientific notation.

```
a = 3.14 # Floating-Point Number
```

```
b = -2.5 # Negative Float
```

```
c = 1.2e3 # Scientific Notation ( $1.2 \times 10^3$ )
```

```
print(type(a)) # Output: <class 'float'>
```

```
print(type(b)) # Output: <class 'float'>
```

```
print(type(c)) # Output: <class 'float'>
```

3. Strings (str)

3.1 What is a String?

A string is a sequence of characters enclosed in single ('), double ("), or triple quotes (''' or ''').

```
text1 = 'Hello'
```

```
text2 = "Python"
```

```
text3 = """Triple quoted strings can span multiple lines."""
```

```
print(type(text1)) # Output: <class 'str'>
```

```
print(type(text2)) # Output: <class 'str'>
```

```
print(type(text3)) # Output: <class 'str'>
```

3.2 Creating Strings

- Single Quotes: 'Hello'
- Double Quotes: "Python"
- Triple Quotes: '''Multi-line String'''

3.3 Accessing Characters in a String

Strings are indexed, meaning each character has a position (starting from 0).

```
text = "Python"
```

```
print(text[0]) # Output: P
```

```
print(text[-1]) # Output: n (Last Character)
```

4. Lists (list)

4.1 What is a List?

A list is an ordered collection of items, which can be of different types. Lists are mutable, meaning they can be changed after creation.

```
fruits = ["Apple", "Banana", "Cherry"]
```

```
numbers = [10, 20, 30]
```

```
print(type(fruits)) # Output:
```

```
print(type(numbers)) # Output:
```

4.2 Accessing Elements in a List

Lists support indexing and slicing.

```
print(fruits[0]) # Output: Apple
```

```
print(fruits[-1]) # Output: Cherry
```

4.3 Modifying a List

Lists allow adding, removing, and modifying elements.

```
fruits.append("Mango")
```

```
print(fruits) # Output: ['Apple', 'Banana', 'Cherry', 'Mango']
```

5. Tuples (tuple)

5.1 What is a Tuple?

A tuple is an immutable collection of elements, meaning its values cannot be changed after creation.

```
colors = ("Red", "Green", "Blue")
```

```
print(type(colors)) # Output:
```

5.2 Accessing Elements in a Tuple

```
print(colors[1]) # Output: Green
```

5.3 Why Use Tuples Instead of Lists?

- Tuples are faster than lists.
- Useful for data that should not be changed (e.g., coordinates, fixed configurations).

6. Sets (set)

6.1 What is a Set?

A set is an unordered collection of unique elements.

```
unique_numbers = {1, 2, 3, 4, 4, 5}
```

```
print(unique_numbers) # Output: {1, 2, 3, 4, 5} (Duplicates Removed)
```

6.2 Set Operations

- Union (|) – Combines two sets.
- Intersection (&) – Returns common elements.
- Difference (-) – Finds elements in one set but not another.

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5}
```

```
print(set1 | set2) # Output: {1, 2, 3, 4, 5} (Union)
```

```
print(set1 & set2) # Output: {3} (Intersection)
```

```
print(set1 - set2) # Output: {1, 2} (Difference)
```

7. Dictionaries (dict)

7.1 What is a Dictionary?

A dictionary is an unordered collection of key-value pairs.

```
student = {"name": "John", "age": 25, "course": "Python"}
```

```
print(type(student)) # Output:
```

7.2 Accessing Values Using Keys

```
print(student["name"]) # Output: John
```

7.3 Modifying a Dictionary

```
student["age"] = 26 print(student) # Output: {'name': 'John', 'age': 26, 'course': 'Python'}
```

7.4 Dictionary Methods

- `.keys()` – Returns all keys.
- `.values()` – Returns all values
- `.items()` – Returns key-value pairs.

```
print(student.keys()) # Output: dict_keys(['name', 'age', 'course'])
```

```
print(student.values()) # Output: dict_values(['John', 26, 'Python'])
```

```
print(student.items()) # Output: dict_items([('name', 'John'), ('age', 26), ('course', 'Python')])
```

8. Built-in Functions for Data Types

8.1 Type Conversion Functions

- `int()` – Converts to an integer.
- `float()` – Converts to a float.
- `str()` – Converts to a string.
- `list()` – Converts to a list.
- `tuple()` – Converts to a tuple.
- `set()` – Converts to a set.
- `dict()` – Converts to a dictionary.

```
num = "100"
```

```
num_int = int(num)
```

```
print(num_int) # Output: 100
```

```
print(type(num_int)) # Output:
```

9. Summary

- Integers (`int`) and floats (`float`) are numeric types.
- Strings (`str`) store text and support indexing.
- Lists (`list`) are mutable and ordered collections.
- Tuples (`tuple`) are immutable and faster than lists.
- Sets (`set`) contain unique, unordered elements with valuable operations.
- Dictionaries (`dict`) store key-value pairs for efficient lookups.
- Python provides built-in functions for the conversion and manipulation of these data types.

Lesson 3: Indexing & Slicing

1. Understanding Indexing

1.1 What is Indexing?

Indexing is a method used to access individual elements in sequential data structures such as strings, lists, and tuples. Python follows zero-based indexing, meaning the first element is at index 0, the second at index 1, and so on.

1.2 Positive Indexing

indexing starts from 0 and moves forward.

```
text = "Python" print(text[0]) # Output: P
```

```
print(text[3]) # Output: h
```

1.3 Negative Indexing

Negative indexing starts from -1, moving backward from the last element.

```
print(text[-1]) # Output: n
```

```
print(text[-3]) # Output: t
```

1.4 Indexing in Lists and Tuples

Lists and tuples also support both positive and negative indexing.

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[1]) # Output: 20
```

```
print(numbers[-2]) # Output: 40
```

1.5 Indexing in Nested Lists

Nested lists (lists within lists) can be accessed using multiple indices.

```
nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
print(nested_list[1][2]) # Output:
```

2. Exploring Slicing

2.1 What is Slicing?

Slicing allows extracting portions of sequences such as strings, lists, and tuples using the start:stop:step syntax.

```
sequence[start:stop:step]
```

2.2 Basic Slicing Operations

- Start – The index from where slicing begins (inclusive).
- Stop – The index where slicing stops (exclusive).
- Step – The increment between indices (default is 1).

```
text = "PythonProgramming"
print(text[0:6]) # Output: Python
print(text[6:]) # Output: Programming (from index 6 to end)
print(text[:6]) # Output: Python (from start to index 5)
```

2.3 Slicing with Step Parameter

The step parameter defines how many indices to skip.

```
numbers = [10, 20, 30, 40, 50, 60, 70]
print(numbers[::2]) # Output: [10, 30, 50, 70] (every second element)
print(numbers[::-1]) # Output: [70, 60, 50, 40, 30, 20, 10] (reversed)
```

3. Indexing and Slicing in Strings

3.1 Extracting Individual Characters

A string behaves like an array where each character has an index.

```
text = "HelloWorld"
print(text[0]) # Output: H
print(text[-1]) # Output: d
```

3.2 Extracting Substrings

Slicing helps extract substrings efficiently.

```
print(text[0:5]) # Output: Hello
print(text[5:]) # Output: World
```

3.3 Using Slicing for String Manipulation

- Reversing a String: `print(text[::-1])` # Output: dlroWolleH
- Skipping Characters: `print(text[::2])` # Output: Hlool
- Extracting the Last Few Characters: `print(text[-3:])` # Output: rld

4. Indexing and Slicing in Lists and Tuples

4.1 Extracting Elements from a List

Lists support indexing just like strings.

```
fruits = ["Apple", "Banana", "Cherry", "Mango", "Orange"]
print(fruits[1]) # Output: Banana
print(fruits[-2]) # Output: Mango
```

4.2 Slicing a List

Slicing allows extracting multiple elements from a list.

```
print(fruits[1:4]) # Output: ['Banana', 'Cherry', 'Mango']
```

```
print(fruits[:3]) # Output: ['Apple', 'Banana', 'Cherry']
```

```
print(fruits[:2]) # Output: ['Apple', 'Cherry', 'Orange']
```

4.3 Modifying Lists Using Indexing and Slicing

Lists are mutable, allowing modification of elements using indexing and slicing.

```
fruits[1] = "Blueberry"
```

```
print(fruits) # Output: ['Apple', 'Blueberry', 'Cherry', 'Mango', 'Orange']
```

Replacing multiple elements at once:

```
fruits[1:3] = ["Grapes", "Peach"]
```

```
print(fruits) # Output: ['Apple', 'Grapes', 'Peach', 'Mango', 'Orange']
```

4.4 Indexing and Slicing in Tuples

Tuples behave like lists but are immutable.

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[2]) # Output: 30
```

```
print(numbers[-1]) # Output: 50
```

```
print(numbers[:3]) # Output: (10, 20, 30)
```

5. Indexing and Slicing in Nested Lists and Tuples

5.1 Extracting Elements from a Nested List

A nested list contains lists within a list.

```
nested_list = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
```

```
print(nested_list[1][2]) # Output: 60
```

```
print(nested_list[2][:2]) # Output: [70, 80]
```

5.2 Extracting Elements from a Nested Tuple

A nested tuple works similarly but remains immutable.

```
nested_tuple = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
```

```
print(nested_tuple[1][1]) # Output: 5
```

```
print(nested_tuple[0][:2]) # Output: (1, 2) 6.
```

Summary

- Indexing is used to access individual elements in sequences using positive (0, 1, 2, ...) or negative (-1, -2, ...) indices.
- Slicing extracts portions of sequences using the start:stop:step syntax.
- Strings, lists, and tuples support both indexing and slicing.
- Lists are mutable, allowing modification using slicing, whereas tuples remain immutable.
- Slicing with step values enables reversing sequences and skipping elements efficiently.
- Nested lists and tuples require multiple indices for deep element access



CodeExpert
Technology, Gadhinglaj.

Lesson 4: Operators in Data Types

1. Introduction to Operators

Operators in Python are used to perform operations on variables and values. They help in arithmetic calculations, comparisons, logical evaluations, variable assignments, and membership checks. Python supports multiple operators categories, including arithmetic, comparison, logical, assignment, and membership operators.

2. Arithmetical Operators

2.1 What are Arithmetical Operators?

Arithmetical operators are used to perform mathematical calculations on numeric values.

2.2 List of Arithmetical Operators

Operator	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulo (Remainder)	$a \% b$
//	Floor Division	$a // b$
**	Exponentiation	$a ** b$

2.3 Examples of Arithmetical Operators

```
a = 15 b = 4 print(a + b) # Output: 19
```

```
print(a - b) # Output: 11
```

```
print(a * b) # Output: 60
```

```
print(a / b) # Output: 3.75
```

```
print(a % b) # Output: 3
```

```
print(a // b) # Output: 3 (Floor Division)
```

```
print(a ** b) # Output: 50625 (Exponentiation)
```

3. Comparison Operators

3.1 What are Comparison Operators?

Comparison operators compare two values and return a boolean result (True or False).

3.2 List of Comparison Operators

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	Less than or equal to

3.3 Examples of Comparison Operators

x = 10

y = 20

print(x == y) # Output: False

print(x != y) # Output: True

print(x > y) # Output: False

print(x < y) # Output: True

print(x >= 10) # Output: True

print(y <= 15) # Output: False

3.4 Real-World Applications of Comparison Operators

- Checking age restrictions (age >= 18)
- Determining higher/lower scores
- Comparing prices in an e-commerce website

4. Logical Operators

4.1 What are Logical Operators?

Logical operators are used to combine multiple conditions and return a boolean value.

4.2 List of Logical Operators

Operator	Description	Example
and	Returns True if both conditions are True	x > 5 and x < 20
or	Returns True if at least one condition is True	x < 5 or x > 20
not	Reverses the result (True to False, False to True)	not (x > 10)

4.3 Examples of Logical Operators

`x = 15`

`print(x > 10 and x < 20) # Output: True`

`print(x > 10 or x < 5) # Output: True`

`print(not(x > 10)) # Output: False`

4.4 Practical Use Cases

- Checking eligibility conditions
- Verifying password strength
- Combining multiple filter conditions in applications

5. Assignment Operators

5.1 What are Assignment Operators?

Assignment operators assign values to variables and modify existing values efficiently.

5.2 List of Assignment Operators

Operator	Description	Example
<code>=</code>	Assigns a value	<code>x = 10</code>
<code>+=</code>	Adds and assigns	<code>x += 5</code> (same as <code>x = x + 5</code>)
<code>-=</code>	Subtracts and assigns	<code>x -= 5</code> (same as <code>x = x - 5</code>)
<code>*=</code>	Multiplies and assigns	<code>x *= 5</code> (same as <code>x = x * 5</code>)
<code>/=</code>	Divides and assigns	<code>x /= 5</code> (same as <code>x = x / 5</code>)
<code>//=</code>	Floor divides and assigns	<code>x //= 5</code> (same as <code>x = x // 5</code>)
<code>%=</code>	Modulo and assigns	<code>x %= 5</code> (same as <code>x = x % 5</code>)
<code>**=</code>	Exponentiation and assigns	<code>x **= 5</code> (same as <code>x = x ** 5</code>)

5.3 Examples of Assignment Operators

`x = 10`

`x += 5`

`print(x) # Output: 15`

`x *= 2`

`print(x) # Output: 30`

`x -= 5`

`print(x) # Output: 25`

5.4 Practical Applications

- Updating scores in a game
- Modifying cart values in e-commerce
- Managing stock inventory

6. Membership Operators

6.1 What are Membership Operators?

Membership operators are used to check whether a value exists in a sequence such as lists, tuples, or strings.

6.2 List of Membership Operators

Operator	Description	Example
in	Returns True if a value exists in a sequence	"apple" in fruits
not in	Returns True if a value does not exist in a sequence	"grape" not in fruits

6.3 Examples of Membership Operators

```
fruits = ["apple", "banana", "cherry"]
```

```
print("banana" in fruits) # Output: True
```

```
print("grape" not in fruits) # Output: True
```

6.4 Real-World Applications

- Checking if a username exists in a database
- Validating if a word is present in a paragraph
- Searching for products in a shopping cart

7. Summary

- Arithmetical operators perform basic mathematical operations such as addition, subtraction, multiplication, and division.
- Comparison operators compare two values and return True or False based on conditions.
- Logical operators combine multiple conditions using and, or, and not.
- Assignment operators help in efficiently modifying variable values.
- Membership operators check if a value exists in lists, tuples, and strings.

Lesson 5: In-Built Functions & Methods in Python

1. Exploring Built-in Functions

1.1 What are Built-in Functions?

Python provides a rich set of built-in functions that perform common operations. These functions are readily available and do not require explicit import statements.

1.2 Commonly Used Built-in Functions

Python's built-in functions can be categorized based on their use cases:

- Numerical Operations: `abs()`, `round()`, `pow()`, `min()`, `max()`, `sum()`
- String Manipulation: `len()`, `ord()`, `chr()`, `str()`, `format()`
- Type Conversion: `int()`, `float()`, `str()`, `list()`, `tuple()`, `set()`, `dict()`
- Data Structure Operations: `sorted()`, `len()`, `zip()`, `reversed()`, `enumerate()`
- Logical Operations: `all()`, `any()`, `bool()`

1.3 Examples of Built-in Functions

Numeric Operations

```
print(abs(-10)) # Output: 10
```

```
print(pow(2, 3)) # Output: 8 (2 raised to the power 3)
```

```
print(round(3.456, 2)) # Output: 3.46 (rounded to 2 decimal places)
```

String Manipulation

```
text = "Python" print(len(text)) # Output: 6
```

```
print(ord('A')) # Output: 65 (ASCII value of A)
```

```
print(chr(97)) # Output: a (Character corresponding to ASCII 97)
```

Type Conversion

```
num = "100"
```

```
num_int = int(num)
```

```
print(num_int) # Output: 100
```

```
print(type(num_int)) # Output:
```

2. Practical Exercises with Functions

2.1 Applying Built-in Functions to Different Data Types

Built-in functions simplify various operations.

Using `sum()` with Lists

```
numbers = [10, 20, 30, 40]
```

```
print(sum(numbers)) # Output: 100
```

Using `sorted()` to Sort Data

```
data = [5, 1, 8, 3]
```

```
print(sorted(data)) # Output: [1, 3, 5, 8]
```

Using `enumerate()` to Get Index and Value

```
fruits = ["Apple", "Banana", "Cherry"]
```

```
for index, fruit in enumerate(fruits):
```

```
print(index, fruit) #
```

```
Output: # 0 Apple # 1 Banana # 2 Cherry
```

3. Understanding Built-in Methods

3.1 What are Built-in Methods?

A method is a function associated with a particular data type. Unlike built-in functions, methods must be called on an object.

3.2 Difference Between Functions and Methods

- Functions are standalone and can work on multiple data types.
- Methods are specific to particular objects (e.g., string methods only work on strings).

4. Built-in Methods for Different Data Types

4.1 String Methods

Strings have several built-in methods for modification and processing.

Common String Methods

Method	Description	Example
<code>.upper()</code>	Converts to uppercase	<code>"hello".upper() → "HELLO"</code>
<code>.lower()</code>	Converts to lowercase	<code>"HELLO".lower() → "hello"</code>
<code>.strip()</code>	Removes whitespace	<code>" hello ".strip() → "hello"</code>
<code>.replace()</code>	Replaces a substring	<code>"hello".replace("l", "r") → "herro"</code>
<code>.split()</code>	Splits into a list	<code>"a,b,c".split(",") → ["a", "b", "c"]</code>

Examples of String Methods

```
text = " Python Programming "
print(text.strip()) # Output: Python Programming
print(text.upper()) # Output: PYTHON PROGRAMMING
print(text.replace("Python", "Java")) # Output: Java Programming
```

4.2 List Methods

Lists support methods for adding, removing, and modifying elements.

Common List Methods

Method	Description	Example
.append()	Adds an element to the list	lst.append(5)
.remove()	Removes the first occurrence	lst.remove(5)
.pop()	Removes an element by index	lst.pop(2)
.sort()	Sorts the list	lst.sort()
.reverse()	Reverses the list	lst.reverse()

Examples of List Methods

```
numbers = [3, 1, 4, 2]
numbers.append(5)
print(numbers) # Output: [3, 1, 4, 2, 5]
numbers.sort() print(numbers) # Output: [1, 2, 3, 4, 5]
```

4.3 Dictionary Methods

Dictionaries store key-value pairs and support various methods for data retrieval and manipulation.

Common Dictionary Methods

Method	Description	Example
.keys()	Returns all keys	dict.keys()
.values()	Returns all values	dict.values()
.items()	Returns key-value pairs	dict.items()
.get()	Retrieves a value	dict.get("name")
.update()	Updates the dictionary	dict.update({"age": 30})

Examples of Dictionary Methods

```
student = {"name": "Alice", "age": 25, "course": "Python"}
print(student.keys()) # Output: dict_keys(['name', 'age', 'course'])
```

```
print(student.values()) # Output: dict_values(['Alice', 25, 'Python'])
```

```
print(student.get("name")) # Output: Alice
```

5. Practical Use of Methods

5.1 Applying Methods to Modify Data Structures

Using built-in methods, we can easily manipulate different data structures.

Updating a Dictionary Using `.update()`

```
student.update({"grade": "A"})
```

```
print(student) # Output: {'name': 'Alice', 'age': 25, 'course': 'Python', 'grade': 'A'}
```

Removing an Element from a List Using `.remove()`

```
numbers = [10, 20, 30, 40]
```

```
numbers.remove(20)
```

```
print(numbers) # Output: [10, 30, 40]
```

6. Best Practices and Efficiency

6.1 Choosing the Right Function or Method

- Use built-in functions whenever possible, as they are optimised for performance.
- Use methods only when working with specific data types.

6.2 Code Optimization and Readability

- Use list comprehensions instead of loops where possible.
- Use dictionary methods like `.get()` instead of direct key access to avoid errors.

6.3 Leveraging Built-in Functions for Efficiency

```
numbers = [3, 6, 1, 9]
```

```
sorted_numbers = sorted(numbers) # More efficient than manual sorting
```

```
print(sorted_numbers) # Output: [1, 3, 6, 9]
```

7. Summary

- Built-in functions perform general operations and work with multiple data types.
- Methods are specific to data types like strings, lists, and dictionaries.
- Python provides efficient built-in functions for numerical calculations, string operations, list manipulations, and dictionary handling.
- Using built-in methods allows easy modification of strings, lists, and dictionaries.

- Choosing the right function or method improves code efficiency and readability.



CodeExpert
Technology, Gadhinglaj.

Chapter 2: Python Control Flow

This chapter focuses on Python's control flow mechanisms, which determine how a program executes statements conditionally or repeatedly. These concepts form the foundation of decision-making and iteration in Python programming

Lesson 1: Statements, Indentation & Conditionals

1. Statements and Indentation

1.1 Understanding Statements in Python

A statement in Python is a single unit of execution that performs an action. Python statements can be categorized as single-line statements, multi-line statements, and compound statements.

1.2 Single-Line Statements

Python allows writing statements in a single line

```
x = 10
```

```
y = 20
```

```
sum_result = x + y
```

```
print(sum_result) # Output: 30
```

1.3 Multi-Line Statements

If a statement is too long to fit in one line, Python allows breaking it across multiple lines using backslashes (\) or parentheses ().

```
total = (10 + 20 + 30 + 40 + 50 + 60)
```

```
print(total) # Output: 210
```

1.4 Python's Indentation Rules

Python uses indentation to define code blocks instead of braces {}. Improper indentation leads to IndentationError.

```
if True:
```

```
    print("Indented correctly") # Output: Indented correctly
```

```
# Incorrect indentation will raise an error:
```

```
# if True:
```

```
    # print("Error") # IndentationError
```

1.5 Importance of Indentation

- Ensures code readability.

- Defines blocks of code in structures like if, for, while, and functions.
- Helps in grouping related statements.

2. Introduction to Conditional Statements

2.1 What are Conditional Statements?

Conditional statements allow a program to execute different blocks of code based on conditions. Python supports three types:

1. if Statement – Executes a block if the condition is True.
2. if-else Statement – Executes one block if True, another if False.
3. if-elif-else Statement – Evaluates multiple conditions sequentially.

2.2 The if Statement

Executes a block of code only if the condition evaluates to True.

```
age = 18
```

```
if age >= 18:
```

```
    print("Eligible to vote") # Output: Eligible to vote
```

2.3 The if-else Statement

Executes one block if True and another if False.

```
num = 10
```

```
if num % 2 == 0:
```

```
    print("Even Number") # Output: Even Number
```

```
else: print("Odd Number")
```

2.4 The if-elif-else Statement

Evaluates multiple conditions in sequence

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade: A")
```

```
elif marks >= 75:
```

```
    print("Grade: B") # Output: Grade: B
```

```
else: print("Grade: C")
```

3. Combining Multiple Conditions

3.1 Using Logical Operators (and, or, not)

Logical operators allow combining multiple conditions.

Operator	Description	Example
and	Returns True if both conditions are True	<code>x > 10 and y < 20</code>
or	Returns True if at least one condition is True	<code>x > 10 or y < 5</code>
not	Reverses the condition	<code>not(x > 10)</code>

3.2 Examples of Logical Operators

```
x = 15
```

```
y = 10
```

```
if x > 10 and y < 20:
```

```
    print("Both conditions are True") # Output: Both conditions are True
```

```
age = 25
```

```
if age < 18 or age > 60:
```

```
    print("Not in working age group")
```

```
else: print("Working age group") # Output: Working age group
```

3.3 Using Parentheses for Clarity

Parentheses ensure correct evaluation order in complex conditions.

```
if (x > 10 and y < 20) or (x == 5 and y > 15):
```

```
    print("Condition met")
```

4. Summary

- Python statements can be single-line or multi-line, and indentation is mandatory.
- Conditional statements (if, if-else, if-elif-else) allow decision-making.
- Logical operators (and, or, not) help combine multiple conditions.

Lesson 2: Loops & Iterations

1. Introduction to Loops

Loops allow repetitive execution of code until a condition is met. Python provides two primary types of loops:

- for loop – Iterates over sequences such as lists, tuples, and strings.
- while loop – Runs based on a condition, executing as long as it remains True.

2. Mastering for Loops

2.1 Understanding the for Loop

A for loop is used when the number of iterations is known or when iterating through a sequence.

2.2 Basic Syntax of a for Loop

variable in sequence:

statement

2.3 Iterating Through Different Sequences

Iterating Over a List

```
fruits = ["Apple", "Banana", "Cherry"]
```

```
for fruit in fruits:
```

```
    print(fruit) # Output: # Apple # Banana # Cherry
```

Iterating Over a String

```
text = "Python" for char in text:
```

```
    print(char) # Output: # P # y # t # h # o # n
```

Iterating Over a Tuple

```
colors = ("Red", "Green", "Blue")
```

```
for color in colors:
```

```
    print(color) # Output: # Red # Green # Blue
```

2.4 Using the range() Function

The range() function generates numerical sequences for iteration.

```
for i in range(5):
```

```
    print(i) # Output: # 0 # 1 # 2 # 3 # 4
```

3. Learning while Loops

3.1 Understanding the while Loop

A while loop executes as long as a given condition is True.

3.2 Basic Syntax of a while Loop

while condition:

statement

3.3 Example of a while Loop

```
count = 1
```

```
while count <= 5:
```

```
    print(count)
```

```
    count += 1
```

```
# Output: # 1 # 2 # 3 # 4 # 5
```

3.4 Using a while Loop with User Input

```
number = int(input("Enter a number greater than 10: "))
```

```
while number <= 10:
```

```
    number = int(input("Try again. Enter a number greater than 10: "))
```

```
print("Correct input received.") # Output depends on user input.
```

4. Comprehending Loop Control Statements

Loop control statements modify the normal flow of loops

Control Statement	Description
break	Exits the loop immediately.
continue	Skips the current iteration and moves to the next.
pass	Acts as a placeholder when no action is needed.

4.1 Using break in a Loop

```
for i in range(10):
```

```
    if i == 5: break
```

```
    print(i) # Output: # 0 # 1 # 2 # 3 # 4
```

4.2 Using continue in a Loop

```
for i in range(5):
```

```
    if i == 2: continue
```

```
    print(i) # Output: # 0 # 1 # 3 # 4
```

4.3 Using pass in a Loop

```
for i in range(3):
    pass # Placeholder statement
```

5. Exploring Nested Loops

5.1 What are Nested Loops?

A nested loop is a loop inside another loop. The inner loop executes for each iteration of the outer loop.

5.2 Example of a Nested Loop

```
for i in range(3):
    for j in range(3):
        print(f"i={i}, j={j}") # Output: # i=0, j=0 # i=0, j=1 # i=0, j=2 # i=1, j=0 # i=1, j=1 # i=1, j=2 # i=2, j=0 #
                                i=2, j=1 # i=2, j=2
```

5.3 Using Nested Loops for Patterns

```
for i in range(1, 6):
    for j in range(i):
        print("*", end=" ")
    print() # Output: # * # * * # * * * # * * * * # * * * * *
```

6. Developing Problem-Solving Skills with Loops

6.1 Summing Numbers Using a Loop

```
numbers = [10, 20, 30, 40]
total = 0
for num in numbers:
    total += num
print("Total Sum:", total) # Output: Total Sum: 100
```

6.2 Finding Even and Odd Numbers

```
for num in range(1, 11):
    if num % 2 == 0:
        print(num, "is Even")
    else: print(num, "is Odd") # Output: # 1 is Odd # 2 is Even # 3 is Odd # 4 is Even # 5 is Odd # 6 is Even
                                # 7 is Odd # 8 is Even # 9 is Odd # 10 is Even
```

6.3 Finding Prime Numbers in a Range

```
for num in range(10, 21):  
    is_prime = True  
    for i in range(2, num):  
        if num % i == 0:  
            is_prime = False  
            break  
    if is_prime:  
        print(num, "is a prime number") # Output: # 11 is a prime number # 13 is a prime number # 17 is a  
        prime number # 19 is a prime number
```

6.4 Looping Through a Dictionary

```
student = {"name": "John", "age": 20, "course": "Python"}  
for key, value in student.items():  
    print(key, ":", value) # Output: # name : John # age : 20 # course : Python
```

7. Summary

- for loops iterate over sequences like lists, tuples, and strings.
- The range() function generates numerical sequences for iteration.
- while loops execute as long as a condition is True.
- Loop control statements (break, continue, pass) modify loop flow.
- Nested loops allow loops inside loops for complex operations.
- Loops are essential for iterating through data structures, generating patterns, and solving problems.

Lesson 3: Conditional & Infinite Looping

1. Introduction to Loops

Loops allow repeated execution of a block of code based on conditions. Instead of manually repeating statements, loops automate repetitive tasks, making code more efficient. Python primarily supports:

- for loops – Used for iterating over sequences.
- while loops – Execute as long as a condition is True.
- Infinite loops – Continue indefinitely unless terminated.

2. Conditional Loops (while Loop)

2.1 Example of a Basic while Loop

```
count = 1
while count <= 5:
    print(count)
    count += 1 # Output: # 1 # 2 # 3 # 4 # 5
```

2.2 Using Conditional Statements Inside while Loops

```
num = 10
while num > 0:
    if num % 2 == 0:
        print(num, "is even")
    num -= 1 # Output: # 10 is even # 8 is even # 6 is even # 4 is even # 2 is even
```

3. Infinite Loops

3.1 What is an Infinite Loop?

An infinite loop continues indefinitely unless a stopping condition is met. This can occur intentionally or due to missing exit conditions.

3.2 Example of an Unintentional Infinite Loop

```
x = 1
while x > 0:
    print(x)
    x += 1 # No stopping condition (runs forever)
```

3.3 Example of an Intentional Infinite Loop

```
while True:
```

```
    user_input = input("Enter 'exit' to stop: ")
```

```
    if user_input == "exit":
```

```
        break
```

Output: The loop runs indefinitely until the user types 'exit'.

3.4 Best Practices to Avoid Infinite Loops

- Ensure loop conditions eventually evaluate to False.
- Use loop control statements (break) to stop when necessary.
- Debug and test loops carefully to prevent unintended infinite execution.

4. Loop Control Statements

4.1 What are Loop Control Statements?

Loop control statements modify the flow of loops by terminating or skipping iterations.

4.2 Using break to Exit a Loop Prematurely

```
for i in range(10):
```

```
    if i == 5: break
```

```
    print(i) # Output: # 0 # 1 # 2 # 3 # 4
```

4.3 Using continue to Skip an Iteration

```
for i in range(5):
```

```
    if i == 2
```

```
        continue
```

```
    print(i) # Output: # 0 # 1 # 3 # 4
```

5. Practical Examples and Exercises

5.1 Finding Even and Odd Numbers Using Loops

```
for num in range(1, 11):
```

```
    if num % 2 == 0:
```

```
        print(num, "is Even")
```

```
    else:
```

```
        print(num, "is Odd") # Output: # 1 is Odd # 2 is Even # 3 is Odd # 4 is Even # 5 is Odd # 6 is Even # 7 is Odd # 8 is Even # 9 is Odd # 10 is Even
```

5.2 Summing Numbers Using a Loop

```
numbers = [10, 20, 30, 40]
total = 0
for num in numbers:
    total += num
print("Total Sum:", total)
```

5.3 Finding Prime Numbers in a Range

```
for num in range(10, 21):
    is_prime = True
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, "is a prime number")
```

Output: # 11 is a prime number # 13 is a prime number # 17 is a prime number # 19 is a prime number

5.4 Looping Through a Dictionary

```
student = {"name": "John", "age": 20, "course": "Python"}
for key, value in student.items():
    print(key, ":", value) # Output: # name : John # age : 20 # course : Python
```

6. Summary

- Conditional loops (while and for) allow repeated execution based on conditions.
- The while loop executes while a condition remains True.
- The for loop iterates over sequences like lists, tuples, and ranges.
- Infinite loops run indefinitely unless explicitly stopped using break.
- Loop control statements (break, continue) alter loop execution flow.
- Loops are essential for iteration, data processing, and solving problems.

Chapter 3: Object-Oriented Programming

This chapter provides a quick revision of the core principles of Object-Oriented Programming (OOP) in Python. OOP allows developers to write modular, reusable, and scalable code by structuring programs around objects and classes.

Lesson 1: Custom Functions in Python

1. Introduction to Custom Functions

1.1 What are Custom Functions?

Custom functions are user-defined blocks of reusable code that perform a specific task. Instead of repeating the same code multiple times, functions help organize, simplify, and reuse logic.

1.2 Benefits of Using Custom Functions

- Code Reusability – Avoids redundancy by defining logic once and calling it multiple times.
- Modularity – Divides complex programs into manageable sections.
- Improved Readability – Makes code cleaner and easier to understand.
- Easier Debugging – Helps identify and fix errors efficiently.

1.3 Real-World Applications of Functions

- Data Processing – Functions are used to clean, filter, and process datasets.
- Mathematical Computation – Used in scientific and financial applications for calculations.
- Web Development – Helps handle user requests, process forms, and manage databases.

2. Defining and Calling Custom Functions in Python

2.1 Syntax for Creating a Function

A function is defined using the `def` keyword, followed by the function name and parentheses `()`.

```
def greet():  
    print("Hello, welcome to Python!")  
  
greet() # Output: Hello, welcome to Python!
```

2.2 Function Parameters

Functions can accept parameters to make them dynamic and flexible.

```
def greet_user(name):  
    print("Hello,", name + "!")  
  
greet_user("Alice") # Output: Hello, Alice!
```

2.3 Calling a Function

Functions are executed by using their name followed by parentheses.

```
def add_numbers(a, b):  
    print("Sum:", a + b)  
  
add_numbers(5, 10) # Output: Sum: 15
```

3. Working with Arguments and Return Values

3.1 Positional Arguments

Arguments are passed in order when calling a function.

```
def full_name(first, last):  
    print("Full Name:", first, last)  
  
full_name("John", "Doe") # Output: Full Name: John Doe
```

3.2 Keyword Arguments

Keyword arguments allow passing values using parameter names.

```
def greet(name, message):  
    print(message, name)  
  
greet(name="Alice", message="Good Morning") # Output: Good Morning Alice
```

3.3 Default Arguments

Default values are assigned to parameters if no value is provided

```
def greet(name="User"):  
    print("Hello,", name + "!!")  
  
greet() # Output: Hello, User!
```

3.4 Returning Values from a Function

The return statement sends a value back to the caller.

```
def square(num):  
    return num * num  
  
result = square(4)  
print(result) # Output: 16
```

3.5 Returning Multiple Values

Functions can return multiple values as tuples

```
def calculate(a, b):
    return a + b, a - b, a * b, a / b

results = calculate(10, 5)

print(results) # Output: (15, 5, 50, 2.0)
```

4. Best Practices for Designing Custom Functions

4.1 Use Meaningful Function and Parameter Names

- **Good Practice:**

```
def calculate_area(length, width):
    return length * width
```

- **Bad Practice:**

```
def ca(x, y):
    return x * y
```

4.2 Use Docstrings for Documentation

A docstring (""" """) provides information about a function.

```
def add(a, b):
    """Returns the sum of two numbers."""
    return a + b
```

4.3 Keep Functions Short and Focused

Each function should perform a single task for better readability.

4.4 Avoid Modifying Global Variables Inside Functions

Using local variables inside functions prevents unintended side effects.

5. Pitfalls to Avoid When Using Custom Functions

5.1 Forgetting to Call a Function

Defining a function without calling it will not execute any code.

```
def hello():
    print("Hello, world!")

# Function not called, so nothing happens.
```

5.2 Forgetting the return Statement

A function without return will output None.

```
def multiply(a, b):
    a * b # Missing return statement
result = multiply(3, 4)
print(result) # Output: None
```

5.3 Infinite Recursion

Recursive functions must have a base condition to prevent infinite calls.

```
def infinite():
    print("This will never stop!")
    infinite() # No stopping condition (infinite recursion)
```

6. Application: Creating Your Own Custom Functions

6.1 Function to Calculate Factorial

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
print(factorial(5)) # Output: 120
```

6.2 Function to Reverse a String

```
def reverse_string(text):
    return text[::-1]
print(reverse_string("Python")) # Output: nohtyP
```

6.3 Function to Find the Maximum Number in a List

```
def find_max(numbers):
    return max(numbers)
print(find_max([10, 25, 32, 18])) # Output: 32
```

6.4 Function to Check if a Number is Prime

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
```

```
return False
```

```
return True
```

```
print(is_prime(11)) # Output: True
```

7. Summary

- Custom functions allow code reuse and modularity.
- A function is defined using `def function_name():` and called using `function_name()`.
- Functions accept arguments (positional, keyword, and default) for flexibility.
- The `return` statement sends values back from a function.
- Best practices include meaningful names, short functions, and proper documentation.
- Avoid forgetting to call functions, missing `return` statements, and infinite recursion



CodeExpert
Technology, Gadhinglaj.